Dynamic Platoon Formation of Multi-Type Autonomous Vehicles for Sustainable Urban Mobility

Jaeyun Ree
Faculty of Information Technology
Monash University
Melbourne, VIC, Australia
jree0010@student.monash.edu

Mohammed Eunus Ali
Faculty of Information Technology
Monash University
Melbourne, VIC, Australia
eunus.ali@monash.edu

***Abstract*—This paper addresses the energy inefficiency of single-occupancy vehicles by introducing a novel cooperative autonomous vehicle system where smaller Passive Vehicles (PVs) can be physically towed by larger Active Vehicles (AVs) during shared highway segments. Unlike traditional platooning that relies solely on aerodynamic drafting, our approach enables complete PV propulsion energy elimination during attached segments. We formulate the platoon formation problem as a constrained optimization problem on a one-dimensional highway model and propose two algorithms: a Greedy Maximum-Weight Matching algorithm and an Iterative Linear Assignment (ILA) algorithm using the Jonker–Volgenant method. A key innovation is the multi-segment matching capability, allowing a single PV to be towed by multiple AVs across different route segments. Experimental evaluation on synthetic scenarios with up to 400 PVs and 80 AVs demonstrates that both algorithms achieve 25–52% distance coverage (proportional to energy savings under constant per-distance propulsion) depending on capacity, with ILA providing per-iteration optimality—i.e., an optimal solution to the bipartite subproblem constructed at each iteration—and superior runtime performance (5–15× faster) due to highly optimized compiled implementations. While global optimality for the full multi-segment problem remains an open question, ILA consistently matches or exceeds greedy performance across all tested scenarios.**

***Index Terms*—autonomous vehicles, vehicle platooning, energy efficiency, linear assignment, Jonker–Volgenant algorithm, combinatorial optimization, intelligent transportation systems**

I. Introduction

Single-occupancy travel remains a major barrier to sustainable urban mobility. U.S. National Household Travel Survey data show that drive-alone commuting dominates trip demand [1], while the transportation sector contributes about 29% of U.S. greenhouse gas emissions [2]. This inefficiency is pronounced when many vehicles traverse overlapping routes but still consume propulsion energy independently.

Figure 1 gives the high-level intuition. Even when many trips overlap, vehicles usually spend propulsion energy independently. If towing-capable vehicles can pull towable vehicles over shared segments, propulsion can be reduced on those covered portions. Because each Activator has a strict capacity limit, only a subset of requests can be served at the same location. Our objective is therefore to find feasible pairings and segment boundaries that maximize total covered towable distance.

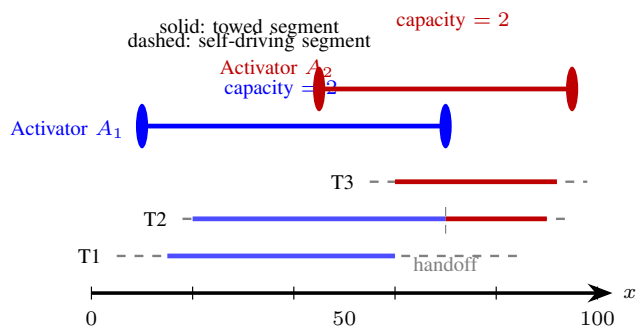


Fig. 1: Conceptual overview of towing-based cooperation. Dashed segments denote self-driving, and solid segments denote towed travel. Activators A_1 and A_2 each have capacity 2, so only feasible segments (including a handoff on $T2$) can be covered.

In the rest of this paper, we call towing-capable vehicles *Active Vehicles (AVs)* and towable vehicles *Passive Vehicles (PVs)*. Unlike drafting-based platooning, our model assumes physical coupling: when a PV is attached, its own propulsion on that segment is fully eliminated. We use covered distance as a proxy for PV-side propulsion-energy reduction, and reserve physics-level load modeling for future work.

Prior work spans platooning [3], [4], ridesharing and routing [6], [7], [9], and assignment optimization [12], [13]. These lines do not model physical towing with both multi-segment PV coverage and point-wise AV capacity constraints in one framework. Our formulation targets that gap.

We study an optimization problem of cooperative towing-based coupling that maximizes covered PV distance under overlap, synchronization, and capacity constraints. For tractability, we evaluate this problem under a one-dimensional highway abstraction. We compare a Greedy Maximum-Weight Matching baseline with an Iterative Linear Assignment

(ILA) method built on SciPy's Jonker–Volgenant implementation [19]. Greedy baseline iteratively selects the currently best feasible AV–PV towing pair (highest benefit) and commits to it. It repeats until no feasible pairs remain, producing a fast but not globally optimal matching.

This paper makes three main contributions. First, we formalize the cooperative towing optimization problem with multi-segment matching and point-wise AV capacity constraints. For tractability, we instantiate and evaluate this formulation on a one-dimensional highway abstraction. Second, we develop and compare two solution strategies: a greedy baseline and an Iterative Linear Assignment (ILA) method based on the Jonker–Volgenant solver [19] with virtual-slot capacity encoding. Third, we provide a controlled experimental study (50–80 AVs, 200–400 PVs) showing substantial coverage savings.

II. Related Work

A. Platooning and Drafting

Platooning studies report about 10–20% fuel savings primarily through aerodynamic drafting [3]–[5]. Vehicles remain physically separated and coordinate spacing/speed. Our setting instead models physical towing, where a covered PV segment does not use its own propulsion. This towing mechanism, plus dynamic attach/detach decisions under capacity, is outside standard drafting formulations.

B. Ridesharing vs. Vehicle–Vehicle Coupling

Ridesharing and dial-a-ride methods optimize passenger-to-vehicle assignment with timing and capacity constraints [6]–[9]. In our problem, the decision variable is vehicle-to-vehicle coupling over road segments. Capacity is point-wise along each towing vehicle's path (attachments vary by position), which differs from conventional seat-capacity constraints.

C. Assignment Solvers and Iterative Gap

Hungarian/Jonker–Volgenant methods solve a one-shot linear assignment optimally for a fixed cost matrix [12], [13], [19]. Our challenge is that feasible gains change after each accepted towing segment because PV residual routes and time states update. Therefore, a single assignment solve is insufficient; we need an iterative solve–update process with virtual-slot capacity encoding.

III. Problem Formulation

We formalize the platoon formation problem on a simplified one-dimensional highway model. This abstraction captures the essential optimization trade-offs while enabling tractable analysis. Extension to general road networks is discussed in Section VI-A.

A. Notation Summary

Table I summarizes the key notation used throughout this paper.

TABLE I: Summary of Notation

Symbol	Description
L	Highway length
$\mathcal{A}$	Set of Active Vehicles (AVs), $\lvert\mathcal{A}\rvert = N$
$\mathcal{P}$	Set of Passive Vehicles (PVs), $\lvert\mathcal{P}\rvert = M$
e_i^a, x_i^a	Entry and exit points of AV a_i
e_j^p, x_j^p	Entry and exit points of PV p_j
C_i	Towing capacity of AV a_i
t_i^a, v_i^a	Entry time and speed of AV a_i
t_j^p, v_j^p	Entry time and speed of PV p_j
$\ell_j, \hat{t}_j$	Current position and arrival time of PV p_j
d_{ij}	Shared travel distance between a_i and p_j
S_{ij}	Energy saving when p_j is towed by a_i
cp_{ij}, dp_{ij}	Coupling and decoupling points
L_{min}	Minimum shared distance for platooning
τ	Time tolerance for coupling synchronization

B. System Model

We consider a coordinated platoon formation system operating on a unidirectional highway segment of length L. The system comprises two distinct vehicle classes.

Definition 1 (Active Vehicle)**.** *An Active Vehicle (AV) $a_i \in \mathcal{A}$ is a larger autonomous vehicle capable of towing multiple smaller vehicles. Each AV is characterized by the tuple $(e_i^a, x_i^a, C_i, t_i^a, v_i^a)$ where $e_i^a, x_i^a \in [0, L]$ are entry and exit points, $C_i \in \mathbb{Z}^+$ is the towing capacity, and t_i^a, v_i^a are the entry time and constant speed.*

Definition 2 (Passive Vehicle)**.** *A Passive Vehicle (PV) $p_j \in \mathcal{P}$ is a smaller autonomous vehicle that can be towed by AVs. Each PV is characterized by the tuple $(e_j^p, x_j^p, t_j^p, v_j^p)$ where $e_j^p, x_j^p \in [0, L]$ are entry and exit points, and t_j^p, v_j^p are entry time and self-driving speed.*

C. Shared Path and Energy Model

For an AV a_i and PV p_j, the *shared path* is the overlap of their routes:

$$d_{ij} = \max(0, \min(x_i^a, x_j^p) - \max(e_i^a, e_j^p)) \tag{1}$$

The coupling point cp_{ij} (where the PV attaches) and decoupling point dp_{ij} (where the PV detaches) are defined as $cp_{ij} = \max(e_i^a, e_j^p)$ and $dp_{ij} = \min(x_i^a, x_j^p)$, respectively.

The energy saving when PV p_j is towed by AV a_i is proportional to shared distance, i.e., $S_{ij} = \alpha d_{ij}$, where α is an energy coefficient. We set $\alpha = 1$, so saving equals covered distance.

D. Multi-Segment Matching

A key innovation of our formulation is *multi-segment matching*: a single PV can be matched to multiple AVs across different segments of its route.

Definition 3 (Segment Assignment)**.** *A segment assignment for PV p_j to AV a_i is a tuple (p_j, a_i, s, e) where $[s, e] \subseteq [e_j^p, x_j^p]$ is the segment during which p_j is towed by a_i.*

For PV p_j, its route from e_j^p to x_j^p can be partitioned into covered segments (towed by AVs, thus saving propulsion

energy) and uncovered segments (self-driven, with no towing-based saving).

Progressive Coverage Assumption: Assignments are processed left-to-right. Each PV tracks a current uncovered boundary ℓ_j (initially e_j^p), and a new segment starts at $\max(e_i^a, \ell_j)$ before advancing ℓ_j to the segment end. Hence uncovered regions remain a contiguous suffix $[\ell_j, x_j^p]$. This design simplifies state management but does not allow back-filling earlier skipped gaps.

E. Optimization Problem

Let $\mathcal{X}$ denote the set of all segment assignments (p_j, a_i, s, e) where PV p_j is towed by AV a_i over segment $[s, e]$. The optimization problem is:

$$\max \sum_{(p_j,a_i,s,e)\in\mathcal{X}} (e-s) \tag{2}$$

$$\text{s.t.} \quad |\{(p_j, a_i, s, e) \in \mathcal{X} : x \in [s, e]\}| \le C_i \quad \forall i \in [N], \forall x \in [e_i^a, x_i^a] \tag{3}$$

$$\text{Segments for each } p_j \text{ are pairwise disjoint} \tag{4}$$

$$(e-s) \ge L_{min} \quad \forall (p_j, a_i, s, e) \in \mathcal{X} \tag{5}$$

Constraint Interpretation:

- **Point-wise Capacity** (3): At any position x along AV a_i's route, the number of simultaneously towed PVs cannot exceed C_i. This is the key constraint that distinguishes our problem from standard bipartite matching—an AV's effective capacity varies along its route as PVs attach and detach.
- **Non-overlap** (4): A PV cannot be towed by two AVs simultaneously at any position.
- **Minimum length** (5): Each towing segment must exceed L_{min} to justify coupling overhead.

Baseline Distance: Without any towing, every PV drives its full route independently. The *baseline total distance* is $B = \sum_{j=1}^{M}(x_j^p - e_j^p)$. The saving percentage is then Saving% $= \frac{\sum(e-s)}{B} \times 100$, which under constant per-distance propulsion equals the percentage of PV propulsion energy eliminated. This baseline serves as the denominator for all efficiency metrics in our experiments.

Lemma 1 (Critical Points Sufficiency)**.** *Fix an AV a_i and let $\mathcal{X}_i = \{(p_j, a_i, s, e) \in \mathcal{X}\}$ be the assignments to a_i. Define the critical set $\mathcal{C}_i = \{e_i^a, x_i^a\} \cup \{s, e : (p_j, a_i, s, e) \in \mathcal{X}_i\}$. Then the point-wise capacity constraint* (3) *holds for all $x \in [e_i^a, x_i^a]$ if and only if it holds for all $x \in \mathcal{C}_i$.*

Proof. The number of PVs towed by AV a_i is a step function over $x \in [e_i^a, x_i^a]$, incrementing at segment start points s and decrementing at segment end points e. Between consecutive critical points, the count is constant. Thus, capacity satisfaction at all critical points implies satisfaction everywhere. □

F. Time Constraints

When temporal synchronization is required:

$$|t_i^a(cp) - t_j^p(cp)| \le \tau \tag{6}$$

where $t_i^a(cp) = t_i^a + (cp - e_i^a)/v_i^a$ is when AV a_i reaches the coupling point cp. For multi-segment matching, the PV arrival time is computed from the current state:

$$t_j^p(cp) = \hat{t}_j + \frac{cp - \ell_j}{v_j^p} \tag{7}$$

where $(\ell_j, \hat{t}_j)$ is the PV's current state. For the first assignment, this reduces to $t_j^p + (cp - e_j^p)/v_j^p$. In our experiments, we use $\tau = 5$ time units with vehicle speeds varying $\pm 20\%$ around a baseline.

Speed Synchronization Policy: Once coupled at the coupling point cp_{ij}, the PV adopts the AV's speed v_i^a for the duration of the towing segment. This policy ensures that feasibility depends solely on synchronization at the coupling point and the existence of a shared route overlap. Decoupling occurs automatically when either vehicle reaches its exit point or when the shared segment ends.

State Update for Multi-Segment Matching: Each PV maintains state $(\ell_j, \hat{t}_j)$ where ℓ_j is the current left boundary of the uncovered interval and $\hat{t}_j$ is the PV's arrival time at position ℓ_j. Initially, $\ell_j = e_j^p$ and $\hat{t}_j = t_j^p$. Let $(\ell_j^{old}, \hat{t}_j^{old})$ denote the state before an assignment. After PV p_j is towed by AV a_i from coupling point cp to decoupling point dp, the state updates as:

$$\ell_j^{new} \leftarrow dp \tag{8}$$

$$\hat{t}_j^{new} \leftarrow \hat{t}_j^{old} + \frac{cp - \ell_j^{old}}{v_j^p} + \frac{dp - cp}{v_i^a} \tag{9}$$

The first term in (9) accounts for independent driving from ℓ_j^{old} to the coupling point, while the second term accounts for towing at the AV's speed.

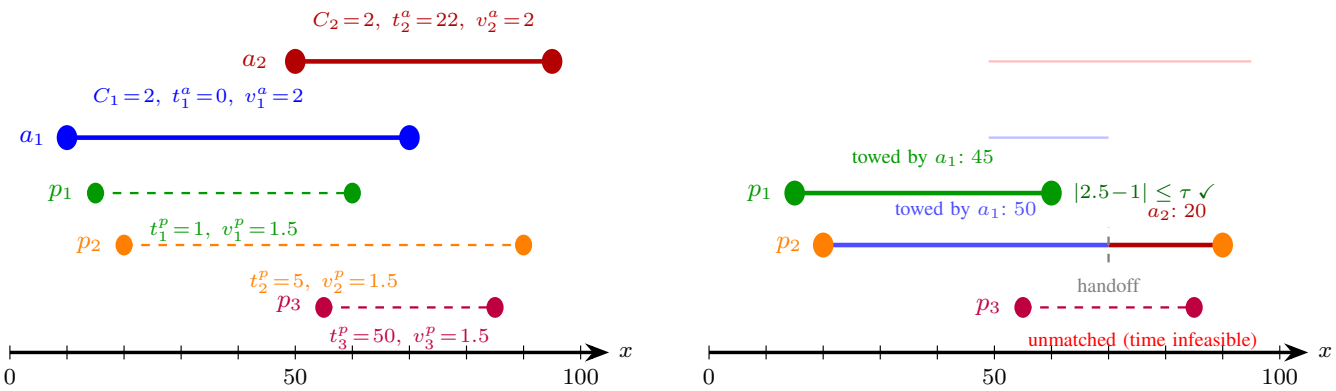
G. Illustrative Example

Figure 2 illustrates a concrete instance with 2 AVs and 3 PVs on a highway of length $L = 100$. Crucially, each vehicle has an entry time and speed, and the time constraint (6) with $\tau = 5$ determines which pairings are feasible. This example demonstrates that spatial overlap alone is insufficient—temporal synchronization at the coupling point is equally decisive.

Example Analysis: With $\tau = 5$, pairings (p_1, a_1) and (p_2, a_1) are time-feasible, while p_3 is infeasible with both AVs and remains unmatched. Multi-segment matching allows p_2 to continue from a_1 to a_2 at the handoff point, increasing its covered distance from 50 to 70. The resulting total saving is 115 out of 145 baseline distance (79.3%).

IV. Proposed Algorithms

We present two algorithms: a greedy baseline and an assignment-based iterative method (ILA). The key modeling point is that this problem is not a one-shot assignment: once a PV is matched for one segment, its remaining uncovered segment changes, and so do later feasible pairings. Therefore, ILA repeatedly rebuilds a gain matrix from the current uncovered PV segments and solves a linear assignment subproblem for that round. AV capacities are encoded by expanding each



(a) Problem instance: vehicle routes with parameters.

(b) Matching result with multi-segment towing. Solid segments denote towed portions, dashed segments denote uncovered self-driving portions, and total towed distances are $p_1 = 45$, $p_2 = 70$, and $p_3 = 0$.

Fig. 2: Illustrative example ($L = 100$, $\tau = 5$) with 2 AVs and 3 PVs. (a) Problem setup with temporal parameters. (b) Matching result: multi-segment matching yields p_2 coverage of 70 units (vs. 50 with single-AV), while p_3 remains unmatched because $|t_i^a(cp) - t_3^p(cp)| > \tau$ for both AVs.

AV into virtual slots before solving. This solve–update loop continues until no feasible assignment remains.

A. Greedy Maximum-Weight Matching

1) Key Insight and Rationale: The greedy approach iteratively selects the single assignment with maximum energy saving, committing immediately before considering alternatives. While intuitive, it may miss globally better solutions due to myopic decisions. We include this algorithm as a *solution-quality baseline*: it provides a natural reference point against which to measure ILA's improvement, requires no external solver library, and is straightforward to implement in resource-constrained environments. Its slower runtime (analyzed below) further motivates the need for batch-based methods such as ILA.

2) Algorithm Description: Algorithm 1 operates in three phases:

Phase 1 (Initialization): For each PV p_j, initialize state $(\ell_j, \hat{t}_j) \leftarrow (e_j^p, t_j^p)$. For each AV, track current assignments for point-wise capacity checking.

Phase 2 (Candidate Generation): Enumerate all feasible (AV, PV, segment) tuples where: (1) overlap with uncovered interval $[\ell_j, x_j^p] \geq L_{min}$, (2) AV has capacity at all critical points, (3) time constraint (6) is satisfied.

Phase 3 (Greedy Selection): Select maximum-saving candidate, update PV and AV states, repeat.

3) Complexity: Let $N = |\mathcal{A}|$ denote the number of AVs and $M = |\mathcal{P}|$ the number of PVs. Each iteration enumerates all feasible (AV, PV) pairs and checks point-wise capacity at each AV's critical points, producing $O(NM)$ candidates. Sorting these candidates requires $O(NM \log(NM))$ time. Because greedy commits to exactly one assignment per iteration, the total number of iterations is bounded by the number of accepted segments, which is at most $O(NM)$ in the worst case (each of M PVs matched to up to N AVs across segments). This yields total worst-case complexity $O((NM)^2 \log(NM))$.

Algorithm 1 Greedy Multi-AV Platoon Matching

Require: AVs $\mathcal{A}$, PVs $\mathcal{P}$, minimum length L_{min}
Ensure: Assignments $\mathcal{X}$, total saving

```
Initialize (ℓ_j, t̂_j) ← (e_j^p, t_j^p) ∀j
Initialize assignments[i] ← ∅ ∀i
X ← ∅, total ← 0
repeat
    candidates ← all feasible (AV, PV, segment) tuples
    if candidates = ∅ then
        break
    end if
    Sort candidates by saving (descending)
    (saving, a_i, p_j, cp, dp) ← candidates[0]
    X ← X ∪ {(p_j, a_i, cp, dp)}
    Update (ℓ_j, t̂_j) via (8)–(9)
    Update assignments[i]
    total ← total + saving
until no candidates
return X, total
```

Why Greedy is Slow in Practice: The key bottleneck is that greedy must *regenerate and re-sort all candidates after every single assignment*. Each accepted pair changes the capacity state of one AV and the coverage state of one PV, potentially invalidating previously feasible candidates. This per-assignment overhead of $O(NM \log(NM))$ accumulates over hundreds of iterations in our experiments. By contrast, ILA processes an entire batch of assignments in one $O(K^3)$ call (where $K = \sum_i C_i$), amortizing overhead across all matches in a round.

B. Iterative Linear Assignment (ILA) Algorithm

1) Key Insight: The linear sum assignment problem (LSAP) seeks a minimum-cost perfect matching in a weighted bipartite

graph and is solvable in $O(n^3)$ time. The classic Hungarian algorithm [12], [13] established this complexity; modern solvers such as SciPy's `linear_sum_assignment` use the Jonker–Volgenant shortest augmenting path method [19], which retains the $O(n^3)$ worst case but achieves substantially faster empirical performance through compiled C code and efficient data structures. Both methods return a provably optimal assignment for any given cost matrix. We adapt LSAP solving via: (1) virtual slot expansion to encode capacity constraints, and (2) iterative application across rounds for multi-segment matching.

2) Virtual Slot Expansion: Expand each AV a_i into C_i virtual slots:

- **Left nodes:** $\sum_{i=1}^{N} C_i$ virtual slots
- **Right nodes:** M PVs (or their current uncovered segments)
- **Edge weight:** $-S_{ij}$ (for minimization formulation)

3) Cost Matrix Construction: The cost matrix $\mathbf{W} \in \mathbb{R}^{(\sum_i C_i)\times M}$ is constructed as follows. For each virtual slot k belonging to AV a_i and each PV p_j:

$$W_{kj} = \begin{cases} -d_{ij}^{curr} & \text{if feasible (overlap } \geq L_{min}\text{, time ok)} \\ +\infty & \text{otherwise} \end{cases} \tag{10}$$

where $d_{ij}^{curr} = \max(0, \min(x_i^a, x_j^p) - \max(e_i^a, \ell_j))$ is the overlap between AV a_i's route and PV p_j's *current uncovered interval* $[\ell_j, x_j^p]$. Feasibility requires: (1) $d_{ij}^{curr} \geq L_{min}$, (2) time synchronization (6) is satisfied, and (3) adding this assignment does not violate capacity at any critical point (Lemma 1).

4) Iterative Algorithm: Algorithm 2 applies ILA iteratively, updating states after each round.

5) Complexity and Implementation: Let $K = \sum_{i=1}^{N} C_i$ denote the total number of virtual slots. Each iteration constructs a $K \times M$ cost matrix in $O(KM)$ time and solves the LSAP in $O(\max(K, M)^3)$ worst-case time. Since each accepted assignment advances at least one PV's boundary, the number of iterations is at most $O(NM)$, giving total worst-case complexity $O(NM \cdot \max(K, M)^3)$.

We use SciPy's `linear_sum_assignment` [19], which implements the Jonker–Volgenant algorithm with highly optimized compiled C code. In practice, ILA is *faster* than Greedy despite the cubic per-iteration cost because: (1) the solver processes all slot–PV pairs in a single batch call with efficient memory access, and (2) the number of ILA iterations is typically small (2–5 in our experiments), whereas Greedy executes hundreds of iterations with $O(NM \log(NM))$ resorting each time.

Capacity Conflict Resolution: When multiple assignments from a single iteration target the same AV, applying them simultaneously may violate point-wise capacity constraints. We resolve this by processing matched pairs sequentially within each iteration: for each $(slot, j)$ pair, we verify that the assignment satisfies capacity at all critical points (Lemma 1) given the *current* state of prior assignments. Assignments that would violate capacity are discarded; the corresponding PVs remain eligible for matching in subsequent iterations. This ensures feasibility while preserving the benefits of batch optimization.

Algorithm 2 Iterative Linear Assignment (ILA) for Multi-AV Matching

Require: AVs $\mathcal{A}$, PVs $\mathcal{P}$, minimum length L_{min}
Ensure: Assignments $\mathcal{X}$, total saving

```
Initialize states as in Algorithm 1
repeat
    Build cost matrix W from current states
    Expand AVs into Σ_i C_i virtual slots
    if no feasible assignments then
        break
    end if
    matching ← LinearSumAssignment(W)
    for each (slot, j) ∈ matching with W_{slot,j} < +∞ do
        if capacity feasible at all critical points (Lemma 1) then
            Accept: X ← X ∪ {(p_j, a_i, cp, dp)}
            Update (ℓ_j, t̂_j) via (8)–(9)
        else
            Discard; p_j remains eligible for next iteration
        end if
    end for
until no new assignments
return X, total
```

C. Theoretical Analysis

Remark on Approximation Guarantees: Under restrictive conditions—single-segment matching only, uniform AV capacity (no point-wise variation), and no time constraints—a greedy algorithm achieves a $\frac{1}{2}$-approximation via reduction to submodular maximization [18]. Our full problem with multi-segment matching, point-wise capacity, and time tolerance does not satisfy these conditions. We therefore position the greedy algorithm as an *empirical solution-quality baseline*: it provides a natural lower bound on achievable savings against which ILA can be evaluated, without claiming formal approximation guarantees.

Theorem 2 (Per-Iteration Optimality)**.** *At each iteration of Algorithm 2, let* $\mathbf{W}$ *be the cost matrix constructed from the current PV states* $\{(\ell_j, \hat{t}_j)\}$ *and AV assignments. The call to* `LinearSumAssignment(`$\mathbf{W}$`)` *returns a matching that maximizes total towed distance among all feasible one-to-one slot-to-PV assignments for that iteration.*

Proof. The cost matrix $\mathbf{W}$ encodes, for every virtual slot–PV pair, the negated towed distance ($-d_{ij}^{curr}$) if the pair is feasible and $+\infty$ otherwise. The LSAP is to find a matching of slots to PVs minimizing total cost. Because the Jonker–Volgenant algorithm solves LSAP exactly [14], the returned matching maximizes total towed distance for the current cost matrix. Convergence follows from the fact that each accepted assignment strictly advances at least one PV's left boundary

ℓ_j, so the number of iterations is bounded by the total number of feasible segments. □

Lemma 3 (Global Optimality Gap)**.** *Algorithm 2 does not guarantee global optimality for the full multi-segment problem* (2)–(5)*. Specifically, assignments accepted in iteration r fix PV states for all subsequent iterations $r' > r$; an alternative set of assignments in iteration r could yield higher total savings globally.*

Proof. Consider two AVs a_1, a_2 and one PV p_1 where a_1 overlaps p_1 on $[0, 30]$ and a_2 overlaps on $[10, 50]$. In the first iteration, ILA may assign $p_1 \to a_1$ (saving 30), advancing ℓ_1 to 30. In the second iteration, the remaining overlap with a_2 is $[30, 50]$ (saving 20), for a total of 50. However, the single assignment $p_1 \to a_2$ on $[10, 50]$ yields 40, and freeing a_1 for another PV could produce a higher global total. The iterative decomposition cannot explore such cross-iteration trade-offs. □

Remark: Despite this gap, our experiments (Section V) show that ILA consistently matches or exceeds greedy performance across all tested configurations. The iterative decomposition is a practical trade-off: it reduces the NP-hard multi-segment problem to a polynomial sequence of tractable LSAP subproblems.

V. Experimental Evaluation

A. Experimental Setup

1) Implementation: Both algorithms are implemented in Python 3.11 using NumPy 1.26 and SciPy 1.11 [19]. The ILA algorithm uses `scipy.optimize.linear_sum_assignment` (Jonker–Volgenant implementation). All experiments run single-threaded on an Apple M4 Pro with 16GB unified memory under macOS 15.0.

2) Scenario Generation: We generate synthetic highway scenarios with the following parameters (Table II):

TABLE II: Experimental Parameters

Parameter	Values	Default
Highway length L	50–1600	100
Number of AVs N	50–80	50
Number of PVs M	200–400	200
AV capacity C	[1,2]–[1,16]	[1,3]
Minimum distance L_{min}	10	10
Time tolerance τ	5.0	5.0
Speed variation	$\pm 20\%$	$\pm 20\%$
Random seeds	42, 43, 44, 45	–

Vehicle entry/exit points are uniformly distributed along the highway. Entry times follow a Poisson process, and speeds vary $\pm 20\%$ around a baseline. All results are averaged over 4 random seeds.

3) Metrics:

- **Baseline Distance**: $\sum_{j=1}^{M}(x_j^p - e_j^p)$, the total distance all PVs would travel independently without any towing. This serves as the denominator for percentage calculations.
- **Total Saving**: Sum of towed distances across all assignments, i.e., $\sum_{(p_j, a_i, s, e) \in \mathcal{X}}(e - s)$.
- **Matched Ratio**: Fraction of PVs receiving at least one assignment.
- **Saving %**: $\frac{\text{Total Saving}}{\text{Baseline Distance}} \times 100$. Under the assumption of constant per-distance propulsion energy, this equals the percentage of PV propulsion energy eliminated.
- **Runtime**: Wall-clock execution time in seconds.

B. Results

1) Capacity Sweep: Table III shows performance as AV capacity varies ($N = 50$, $M = 200$, $L = 100$).

TABLE III: Performance vs. AV Capacity (averaged over 4 seeds)

	Greedy			ILA		
C	Save	%	Time(s)	Save	%	Time(s)
$[1, 2]$	1946	24.5	0.23	1974	24.8	**0.02**
$[1, 4]$	2709	34.0	0.32	2725	34.2	**0.04**
$[1, 8]$	3556	44.7	0.43	3625	45.7	**0.08**
$[1, 16]$	4091	51.4	0.45	4159	52.3	**0.16**

Key Observations:

- ILA achieves 0.3–1.9% higher savings while being 2.8–11.5× faster
- Both algorithms scale linearly with capacity
- Saving % improves from 24.5% to 52.3% as capacity increases from 2 to 16

2) Highway Length Sweep: Table IV shows performance as highway length varies ($N = 80$, $M = 400$, $C = [1, 3]$).

TABLE IV: Performance vs. Highway Length (averaged over 4 seeds)

	Greedy			ILA		
L	Save	%	Time(s)	Save	%	Time(s)
50	2996	31.4	1.98	3017	31.6	**0.07**
100	4594	29.8	1.59	4656	30.7	**0.11**
200	7423	26.8	1.46	7474	27.0	**0.14**
400	10742	21.2	1.39	10784	21.2	**0.15**
800	13673	13.9	0.93	13715	13.9	**0.11**
1600	14385	7.4	0.52	14481	7.5	**0.10**

Key Observations:

- Saving % decreases with highway length (sparser vehicle overlap)
- ILA is 5–28× faster across all lengths
- Absolute savings increase but efficiency drops for longer highways

3) Baseline Distance Comparison: Figure 3 shows the total PV distance under three conditions: (1) *Baseline*—no towing, every PV drives independently; (2) *After Greedy*—remaining distance after greedy matching; (3) *After ILA*—remaining distance after ILA matching. The gap between Baseline and the optimized bars directly visualizes the energy eliminated by towing.

4) Algorithm Comparison Visualization: Figures 4 and 5 show saving percentage comparisons graphically.

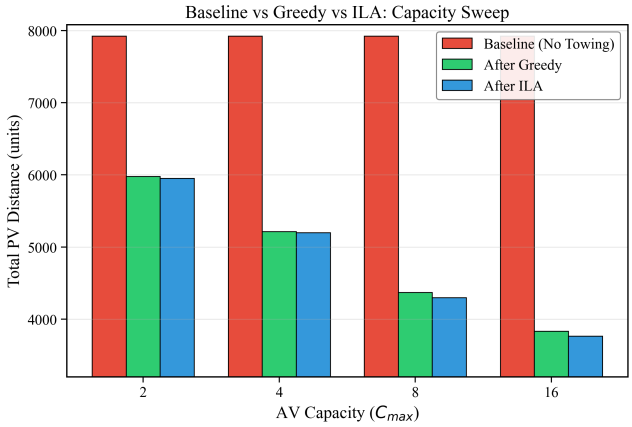


Fig. 3: Total PV distance: Baseline (no towing) vs. after Greedy vs. after ILA across AV capacity values. The difference between Baseline and each algorithm represents total distance saved (proportional to PV propulsion energy eliminated).

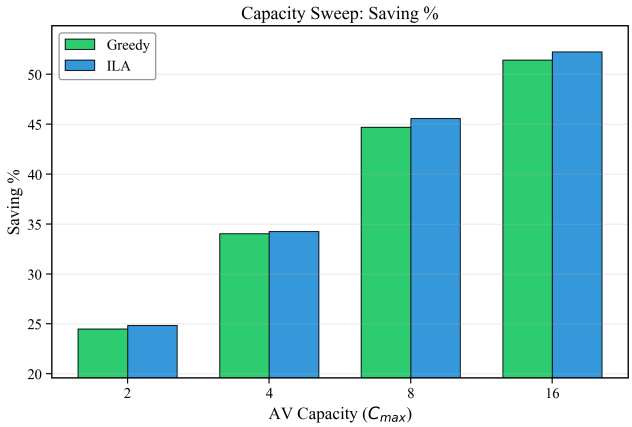


Fig. 4: Distance saving percentage vs. AV capacity for both algorithms. ILA consistently matches or exceeds greedy while being faster.

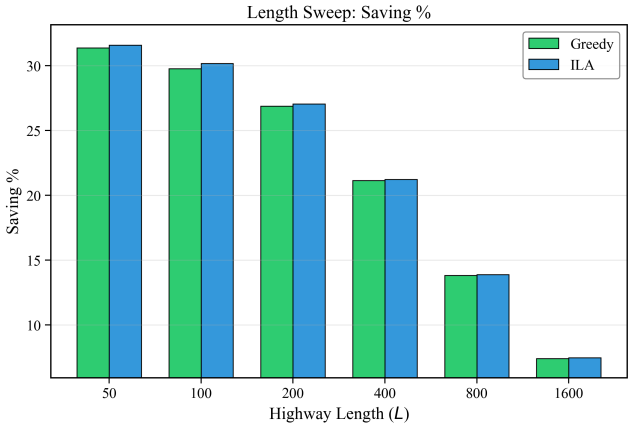


Fig. 5: Distance saving percentage vs. highway length. Both algorithms show decreasing efficiency as highways lengthen due to sparser vehicle overlap.

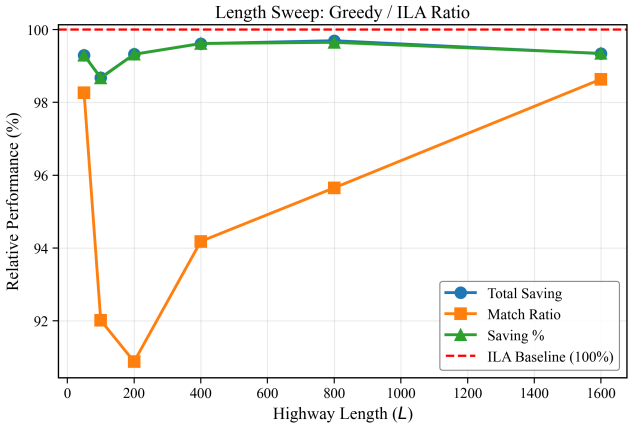


Fig. 6: Relative performance (ILA/Greedy ratio) across highway lengths. Values <1 indicate ILA is faster and/or achieves higher savings.

5) Scalability and Runtime Analysis: Figures 6 and 7 show runtime comparisons across scenarios.

C. Discussion

Why is ILA Faster Despite Higher Worst-Case Complexity? Despite the $O(K^3)$ per-iteration cost of solving a linear assignment problem (where $K = \sum_i C_i$), ILA outperforms Greedy in wall-clock time. This is due to:

1) `scipy.optimize.linear_sum_assignment` uses highly optimized compiled C code with efficient memory access patterns
2) Greedy requires repeated sorting ($O(NM \log NM)$ per iteration) and candidate regeneration after each assignment
3) ILA processes all assignments in batches, amortizing overhead

Practical Implications: ILA is the recommended algorithm for deployment—it provides both per-iteration optimality and superior speed. The greedy algorithm serves two roles: (1) as a solution-quality reference that validates ILA's empirical performance (ILA matches or exceeds greedy savings in every configuration), and (2) as a dependency-free alternative for constrained environments lacking compiled LSAP solvers.

Capacity Impact: Doubling capacity yields sub-linear improvement (24.8% $\rightarrow$ 52.3%), suggesting diminishing returns. Fleet operators should balance capacity costs against marginal savings.

Length Effect: Saving percentage drops from 31.6% (L=50) to 7.5% (L=1600) as vehicles have less overlap on longer highways. This suggests our approach is most effective for medium-length corridors with high vehicle density.

D. Limitations

Our results should be interpreted within four scope boundaries: (1) a one-dimensional highway abstraction rather than a full road network, (2) a distance-based saving proxy instead of a physics-level power model, (3) per-iteration optimality for

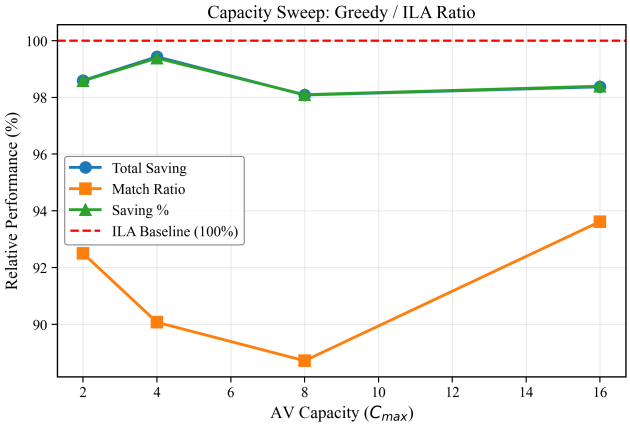


Fig. 7: Relative performance (ILA/Greedy ratio) across capacity values. ILA maintains consistent speedup (2.8–11.5×) across all capacity settings.

ILA without global optimality guarantees for the full multi-segment objective, and (4) an offline setting with complete trajectory knowledge. These assumptions keep the analysis tractable and define the applicability of the reported performance.

VI. Conclusion

We presented a dynamic towing-based platoon formation framework for heterogeneous autonomous vehicles and formulated it as a constrained multi-segment matching problem on a one-dimensional corridor. Across scenarios with up to 80 AVs and 400 PVs, both algorithms achieved substantial coverage gains (25–52%), while ILA consistently matched or exceeded greedy solution quality and delivered 5–15× faster runtime through batch assignment with an optimized Jonker–Volgenant solver. The framework captures point-wise capacity and temporal synchronization constraints, and the theoretical scope is explicit: ILA guarantees per-iteration optimality for each LSAP subproblem but not global optimality for the full multi-segment objective.

A. Future Work

Future work will extend the model to two-dimensional road networks, evaluate online and uncertain traffic settings with periodic re-optimization, and investigate decentralized coordination and incentive mechanisms for large-scale deployment. We will also validate the distance-based saving proxy against physics-based energy models and practical coupling constraints using real-world trajectory datasets and AV/PV platform studies.

References

[1] U.S. Department of Transportation, Federal Highway Administration, "Summary of Travel Trends: 2022 National Household Travel Survey," Report FHWA-HPL-24-009, 2024.

[2] U.S. Environmental Protection Agency, "Inventory of U.S. Greenhouse Gas Emissions and Sinks: 1990–2021," EPA 430-R-23-003, 2023.

[3] S. Tsugawa, S. Jeschke, and S. E. Shladover, "A Review of Truck Platooning Projects for Energy Savings," *IEEE Trans. Intell. Veh.*, vol. 1, no. 1, pp. 68–77, 2016.

[4] C. Bergenhem, H. Pettersson, E. Coelingh, C. Englund, S. Shladover, and S. Tsugawa, "Overview of platooning systems," in *Proc. 19th ITS World Congress*, Vienna, Austria, 2012.

[5] S. E. Shladover, "Cooperative (rather than autonomous) vehicle-highway automation systems," *IEEE Intell. Transp. Syst. Mag.*, vol. 1, no. 1, pp. 10–19, 2009.

[6] M. Furuhata, M. Dessouky, F. Ordóñez, M.-E. Brunet, X. Wang, and S. Koenig, "Ridesharing: The state-of-the-art and future directions," *Transport. Res. Part B: Methodol.*, vol. 57, pp. 28–46, 2013.

[7] N. Agatz, A. Erera, M. Savelsbergh, and X. Wang, "Optimization for dynamic ride-sharing: A review," *European J. Oper. Res.*, vol. 223, no. 2, pp. 295–303, 2012.

[8] J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli, and D. Rus, "On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment," *Proc. Natl. Acad. Sci. U.S.A.*, vol. 114, no. 3, pp. 462–467, 2017.

[9] J.-F. Cordeau and G. Laporte, "The dial-a-ride problem: models and algorithms," *Annals Oper. Res.*, vol. 153, no. 1, pp. 29–46, 2007.

[10] A. Sciarretta and A. Vahidi, *Energy-Efficient Driving of Road Vehicles*, Springer, 2020.

[11] E. Hellström, M. Ivarsson, J. Åslund, and L. Nielsen, "Look-ahead control for heavy trucks to minimize trip time and fuel consumption," *Control Eng. Practice*, vol. 17, no. 2, pp. 245–254, 2009.

[12] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Res. Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[13] J. Munkres, "Algorithms for the assignment and transportation problems," *J. Soc. Ind. Appl. Math.*, vol. 5, no. 1, pp. 32–38, 1957.

[14] R. Burkard, M. Dell'Amico, and S. Martello, *Assignment Problems*, SIAM, 2012.

[15] D. G. Cattrysse and L. N. Van Wassenhove, "A survey of algorithms for the generalized assignment problem," *European J. Oper. Res.*, vol. 60, no. 3, pp. 260–272, 1992.

[16] J. Ploeg, B. T. M. Scheepers, E. van Nunen, N. van de Wouw, and H. Nijmeijer, "Design and experimental evaluation of cooperative adaptive cruise control," in *Proc. IEEE Int. Conf. Intell. Transp. Syst.*, pp. 260–265, 2011.

[17] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *J. Artif. Intell. Res.*, vol. 31, pp. 591–656, 2008.

[18] M. L. Fisher, G. L. Nemhauser, and L. A. Wolsey, "An analysis of approximations for maximizing submodular set functions—II," *Math. Programming Study*, vol. 8, pp. 73–87, 1978.

[19] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[20] M. Drexl, "Synchronization in vehicle routing—a survey of VRPs with multiple synchronization constraints," *Transport. Sci.*, vol. 46, no. 3, pp. 297–438, 2012.

[21] R. M. Karp, U. V. Vazirani, and V. V. Vazirani, "An optimal algorithm for on-line bipartite matching," in *Proc. 22nd Ann. ACM Symp. Theory of Computing*, pp. 352–358, 1990.

[22] W. Ren and R. W. Beard, "Consensus seeking in multiagent systems under dynamically changing interaction topologies," *IEEE Trans. Autom. Control*, vol. 50, no. 5, pp. 655–661, 2005.